Spenny Piggy – Launch Master Implementation Document (Ultra Detailed)

Stripe-safe launch & scale to 1,500 creators • Generated 2026-02-26

This is the full detailed build document (not a summary). It contains: architecture, exact rule logic, table schemas, endpoints, Stripe example code, webhook handlers, pop-up copy, cron SQL, and staging stress-test checklists.

Non■negotiable: Risk Engine runs **before** Stripe PaymentIntent creation. If rule says BLOCK → no PaymentIntent is created.

Glossary (Simple English)

Step■Up = Extra check before payment. (OTP code + “type your name”).

Cool■Down = Short pause. (User must wait 15 minutes).

Review Hold = Payment succeeds, but payout is held until safe.

Block = Payment is not allowed. No Stripe PaymentIntent created.

Throttle = System slows down spending & onboarding when risk rises.

- Goal: stop fast/impulsive spending and cross■creator clusters that become disputes.
- Founder goal: sleep. In the morning, open dashboard and review a short action list.

1. System Architecture (Mandatory Flow)

All payments must follow this exact order:

```
Supporter (Guest/Registered)
→ Frontend Checkout
→ Backend Risk Engine (ALLOW / STEP_UP / COOLDOWN / REVIEW_HOLD / BLOCK)
→ Stripe PaymentIntent (only if allowed)
→ Stripe Radar + 3DS
→ Stripe Webhooks
→ Internal Ledger Update
→ Rolling Metrics Update
→ Friday Payout Engine
→ Creator Payout
```

Rule: If Risk Engine returns BLOCK → do not call Stripe.

Implementation Notes

- Risk Engine must be called from a single backend endpoint used by all clients.
- No direct frontend-to-Stripe creation of PaymentIntents is allowed.
- All webhook handlers must be idempotent (safe to receive same event twice).
- Every automation decision must write to audit_logs (no silent actions).

2. Stripe Foundation (Connect + Safe Funds Flow)

Objective

- Use Stripe Connect (Express or Standard). Each creator has a connected account.
- Use either: (A) destination charges, or (B) separate charges + transfers (choose one and standardise).
- Store platform fee and apply it consistently.
- Webhooks must be subscribed and verified (signature).

Recommended approach (destination charge pattern)

This keeps payment creation simple and ensures money routes to the creator account, with platform fee retained.

```
// Node.js (Stripe SDK) – destination charge example
const stripe = require('stripe')(process.env.STRIPE_SECRET_KEY);

async function createPI({amount, currency, connectedAccountId, feeAmount, metadata}) {
  const pi = await stripe.paymentIntents.create({
    amount, // in minor units (pence)
    currency,
    application_fee_amount: feeAmount, // platform fee in minor units
    transfer_data: { destination: connectedAccountId },
    automatic_payment_methods: { enabled: true },
    metadata,
  });
  return pi;
}
```

3DS forcing when required

```
// You can force 3DS by using request_three_d_secure
const pi = await stripe.paymentIntents.create({
  amount,
  currency: 'gbp',
  payment_method_types: ['card'],
  payment_method_options: {
    card: { request_three_d_secure: 'any' } // force 3DS challenge when possible
  },
  application_fee_amount: feeAmount,
  transfer_data: { destination: connectedAccountId },
});
```

Acceptance Criteria

- A test payment routes funds to the correct connected account.
- Platform fee is retained correctly.
- Disputes are visible and handled correctly in Stripe.
- Webhook signature verification is enabled and required.

3. Database Schemas (Required Tables)

Use Postgres examples below. If you use another DB, match fields and behaviour exactly.

3.1 platform_risk_state

```
CREATE TABLE platform_risk_state (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  state TEXT NOT NULL CHECK (state IN ('NORMAL','CAUTION','THROTTLE','FREEZE')),
  reason_codes TEXT[] NOT NULL DEFAULT '{}',
  reason_detail TEXT NULL,
  set_by TEXT NOT NULL CHECK (set_by IN ('system','admin')),
  set_by_user_id UUID NULL,
  started_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  expires_at TIMESTAMPTZ NULL,
  metrics_snapshot JSONB NOT NULL DEFAULT '{}'::jsonb,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_platform_risk_state_started_at ON platform_risk_state (started_at DESC);
```

3.2 risk_identity

```
CREATE TABLE risk_identity (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  -- identity keys (store hashes)
  card_fingerprint TEXT NULL,
  email_hash TEXT NULL,
  device_id_hash TEXT NULL,
  ip_hash TEXT NULL,

  is_guest BOOLEAN NOT NULL DEFAULT true,
  trust_tier INT NOT NULL DEFAULT 0, -- 0/1/2

  cooldown_until TIMESTAMPTZ NULL,
  is_blocked BOOLEAN NOT NULL DEFAULT false,

  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

3.3 identity_rollups (fast rolling counters)

```
CREATE TABLE identity_rollups (
  risk_identity_id UUID PRIMARY KEY REFERENCES risk_identity(id),

  spend_10m BIGINT NOT NULL DEFAULT 0,
  spend_1h BIGINT NOT NULL DEFAULT 0,
  spend_2h BIGINT NOT NULL DEFAULT 0,
  spend_24h BIGINT NOT NULL DEFAULT 0,
  spend_48h BIGINT NOT NULL DEFAULT 0,
  spend_7d BIGINT NOT NULL DEFAULT 0,

  payment_count_10m INT NOT NULL DEFAULT 0,

  creators_paid_24h INT NOT NULL DEFAULT 0,
  creators_paid_48h INT NOT NULL DEFAULT 0,
  new_creators_24h INT NOT NULL DEFAULT 0,

  disputes_30d INT NOT NULL DEFAULT 0,

  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

3.4 payments

```
CREATE TABLE payments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  creator_id UUID NOT NULL,
  risk_identity_id UUID NOT NULL REFERENCES risk_identity(id),
  amount BIGINT NOT NULL, -- minor units
  currency TEXT NOT NULL DEFAULT 'gbp',
  stripe_payment_intent_id TEXT NULL,
```

```
  status TEXT NOT NULL CHECK (status IN (
    'initiated','step_up','review_hold','succeeded','refunded','disputed','blocked','failed'
  )),

  confirmation_log_id UUID NULL,
  reason_codes TEXT[] NOT NULL DEFAULT '{}',

  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

3.5 confirmation_logs

```
CREATE TABLE confirmation_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  payment_id UUID NULL,
  risk_identity_id UUID NOT NULL REFERENCES risk_identity(id),

  ip_hash TEXT NULL,
  device_id_hash TEXT NULL,

  otp_verified BOOLEAN NOT NULL DEFAULT false,
  typed_confirmation TEXT NULL,

  spend_snapshot JSONB NOT NULL DEFAULT '{}'::jsonb,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

3.6 disputes + early fraud warnings

```
CREATE TABLE disputes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  payment_id UUID NULL,
  creator_id UUID NULL,
  stripe_dispute_id TEXT NOT NULL,
  amount BIGINT NOT NULL,
  currency TEXT NOT NULL DEFAULT 'gbp',
  reason TEXT NULL,
  status TEXT NOT NULL DEFAULT 'open', -- open/won/lost
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  resolved_at TIMESTAMPTZ NULL
);

CREATE TABLE early_fraud_warnings (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  payment_id UUID NULL,
  stripe_efw_id TEXT NOT NULL,
  stripe_charge_id TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

3.7 creator_metrics + payout

```
CREATE TABLE creator_metrics (
  creator_id UUID PRIMARY KEY,
  tx_30d INT NOT NULL DEFAULT 0,
  disputes_30d INT NOT NULL DEFAULT 0,
  dispute_rate_30d NUMERIC(6,3) NOT NULL DEFAULT 0.000, -- percent
  refunds_30d INT NOT NULL DEFAULT 0,
  refund_rate_30d NUMERIC(6,3) NOT NULL DEFAULT 0.000, -- percent

  reserve_percent INT NOT NULL DEFAULT 0, -- 0/5/10/15
  payout_delay_days INT NOT NULL DEFAULT 0,

  top_buyer_percent NUMERIC(6,3) NOT NULL DEFAULT 0.000, -- concentration
  volatility_score INT NOT NULL DEFAULT 0,

  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE payout_runs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  run_date DATE NOT NULL,
  status TEXT NOT NULL DEFAULT 'preview', -- preview/executed/failed
  totals JSONB NOT NULL DEFAULT '{}'::jsonb,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE TABLE audit_logs (
```

```
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  actor TEXT NOT NULL, -- system/admin
  action_type TEXT NOT NULL,
  reference_id TEXT NULL,
  metadata_json JSONB NOT NULL DEFAULT '{}'::jsonb,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

4. Required Backend Endpoints (API Contract)

Core endpoints

- POST /risk/evaluate → returns decision + reasons + UI message keys
- POST /payments/create-intent → calls /risk/evaluate then creates PI only if allowed
- POST /webhooks/stripe → handles all Stripe events (verified signatures)
- GET /admin/risk-state → latest platform state
- POST /admin/risk-state → manual set state (admin only) + audit log
- POST /admin/kill-switch → enable/disable toggles (guest disable, force step-up, etc.)
- GET /admin/dashboard → founder morning summary + action queue
- POST /admin/payout/preview → compute Friday payouts
- POST /admin/payout/execute → execute payouts
- GET /admin/export/audit-pack → Stripe-ready export bundle

Exact JSON response from /risk/evaluate

```
{
  "decision": "STEP_UP",
  "reason_codes": ["ACCELERATION_3_IN_10M"],
  "limits": {
    "max_spend_1h": 100000,
    "max_spend_24h": 250000,
    "max_new_creators_24h": 1,
    "cooldown_minutes": 15,
    "review_hold_threshold": 150000
  },
  "ui": {
    "supporter_message_key": "STEP_UP_REQUIRED",
    "supporter_message_title": "Confirm Your Payment",
    "supporter_message_body": "For your security, please confirm this payment."
  }
}
```

Acceptance Criteria

- All clients use /payments/create-intent (no bypass).
- Risk decision is returned with clear reason_codes for auditability.
- Every request produces an audit log entry (decision + reason + identity + amount + creator).

5. getEffectiveLimits() (One Place For All Limits)

Why

Developers must NOT hardcode limits in many places. There must be one function that returns the current limits based on platform state + identity type.

Function behaviour

- Input: risk_identity (guest/registered, trust_tier), platform state (NORMAL/CAUTION/THROTTLE/FREEZE)
- Output: all limits used by risk engine and onboarding engine
- Used by: /risk/evaluate, onboarding activation, admin dashboards

Example JSON outputs (per platform state)

```
// NORMAL (registered)
{"max_spend_1h":200000,"max_spend_24h":500000,"max_spend_7d":1000000,"max_new_creators_24h":1,
 "step_up_threshold":150000,"cooldown_minutes":15,"review_hold_threshold":250000,"guest_allowed":true}

// CAUTION
{"max_spend_1h":100000,"max_spend_24h":250000,"max_spend_7d":500000,"max_new_creators_24h":1,
 "step_up_threshold":50000,"cooldown_minutes":15,"review_hold_threshold":150000,"guest_allowed":true}

// THROTTLE
{"max_spend_1h":75000,"max_spend_24h":150000,"max_spend_7d":300000,"max_new_creators_24h":1,
 "step_up_threshold":25000,"cooldown_minutes":30,"review_hold_threshold":100000,"guest_allowed":false}

// FREEZE (payments still possible, but strict + no onboarding)
{"max_spend_1h":50000,"max_spend_24h":100000,"max_spend_7d":200000,"max_new_creators_24h":1,
 "step_up_threshold":25000,"cooldown_minutes":30,"review_hold_threshold":75000,"guest_allowed":false}
```

Acceptance Criteria

- Changing platform_risk_state changes returned limits immediately.
- All risk decisions use these limits (single source of truth).

6. Rule: 3 payments in 10 minutes → STEP-UP

IF this happens…

- Count payments per risk_identity in rolling 10 minutes (payment_count_10m).
- If payment_count_10m >= 3 → trigger STEP_UP before PaymentIntent creation.

DO this…

- Show OTP popup + require typed confirmation.
- Create confirmation_logs row and link to payment attempt.
- Only proceed to Stripe if OTP is verified.

Which means…

- User sees: ‘Confirm your payment’.
- If they fail or cancel, payment stops.
- We store proof for dispute evidence.

Developers must build

- Rolling counter update (identity_rollups.payment_count_10m).
- OTP service (email/SMS) + verification endpoint.
- Confirmation log storage (ip_hash, device_id_hash, spend snapshot).
- Audit log with reason_code=ACCELERATION_3_IN_10M.

Test in staging

- Simulate same identity making 3 payments quickly.
- Verify step-up triggers on 3rd attempt.
- Verify no PaymentIntent without OTP success.

Acceptance Criteria

- Step-up always triggers on rule.
- Confirmation log exists and is linked.
- Audit log exists for each decision.

7. Rule: Continued rapid payments after STEP-UP → COOLDOWN 15 minutes

IF this happens…

- If identity triggered STEP_UP within last 15 minutes AND continues to attempt new payments rapidly.
- Or if payment_count_10m continues above threshold after step-up.

DO this…

- Set risk_identity.cooldown_until = now() + 15 minutes.
- Return COOLDOWN and do not create PaymentIntent.

Which means…

- User sees: ‘Please wait 15 minutes’.
- System pauses them automatically then unlocks after time.

Developers must build

- Cooldown field and check at start of /risk/evaluate.
- Popup message key COOLDOWN_ACTIVE.
- Audit log with reason_code=COOLDOWN_AFTER_STEP_UP.

Test in staging

- Attempt payment during cooldown.
- Verify PaymentIntent not created.
- After 15 minutes, verify payment can proceed.

Acceptance Criteria

- No PaymentIntent during cooldown.
- Cooldown auto expires.
- Audit log recorded.

8. Rule: >£7,500 in 2 hours → Force 3DS + STEP-UP (and possible Review Hold)

IF this happens…

- Track spend_2h per identity (identity_rollups.spend_2h).
- If spend_2h >= 750000 (pence) → trigger.

DO this…

- Require STEP_UP.
- Create PaymentIntent with request_three_d_secure='any'.
- If behaviour continues, mark future high-value payments as REVIEW_HOLD.

Which means…

- User must pass extra checks.
- Stripe sees stronger authentication.
- We prevent post-payout exposure by holding risky payments.

Developers must build

- Accurate spend_2h rolling calculation.
- Stripe PI creation code supports forcing 3DS.
- Review_hold routing logic in risk engine based on repeated triggers.

Test in staging

- Simulate spending pattern crossing threshold.
- Verify 3DS request is set on PI.
- Verify STEP_UP required.

Acceptance Criteria

- 3DS forced when rule triggers.
- OTP required and logged.
- If review_hold set, payout excludes it.

9. Rule: Exceed new-creator-per-day limit → BLOCK

IF this happens…

- Count ‘new’ creators (first-time paid) per identity in last 24h (new_creators_24h).
- Default Tier 0: 1 new creator per 24h.

DO this…

- Return BLOCK and do not create PaymentIntent.
- Allow repeat payments to creators already paid before.

Which means…

- User sees: ‘You can support 1 new creator per day. You can still pay creators you paid before.’

Developers must build

- Maintain table or cached relation identity↔creator to know ‘already paid’.
- Update identity_rollups.new_creators_24h.
- Audit log reason_code=NEW_CREATOR_LIMIT.

Test in staging

- Attempt pay creator A (new) then creator B (new) within 24h.
- Second attempt must be blocked.

Acceptance Criteria

- Second new creator payment is blocked.
- Repeat payment to creator A is allowed.

10. Rule: Cross-creator spend > £5k in 48h → restrict NEW creators 24–72h

IF this happens…

- Compute spend_48h and creators_paid_48h across distinct creators.
- If spend_48h > 500000 and creators_paid_48h >= 2 → trigger.

DO this…

- Set a restriction flag (e.g., risk_identity.cooldown_until or separate new_creator_restrict_until).
- Block new creator payments, allow existing creator payments.

Which means…

- This stops ‘whale hopping’ across creators (big Stripe risk).

Developers must build

- Track distinct creators in 48h rollup.
- Implement restriction window (24–72h).
- UI copy for restriction.

Test in staging

- Simulate identity paying 2 creators and exceeding £5k within 48h.
- Try pay 3rd new creator — must be blocked.
- Try repay existing creator — must be allowed.

Acceptance Criteria

- New creator payments blocked for duration.
- Existing creator payments allowed.
- Audit logs for trigger and enforcement.

11. Platform Throttle States (Automatic System Brake)

States and what they do

State	Meaning	What changes automatically
NORMAL	All good	Default limits; onboarding batches normal
CAUTION	Be careful	Lower spend caps; more step-up; monitor spikes
THROTTLE	Slow down	Lower caps; more cooldowns; disable guest; slower onboarding
FREEZE	Stop onboarding	No new creator activation; strict caps; guest disabled

Automatic triggers (cron jobs)

- Daily GMV > 150% of 7-day avg → CAUTION
- Weekly GMV > 130% of previous week → THROTTLE
- Platform dispute rate (30d) ≥ 0.7% → FREEZE
- 5+ creators ≥ 0.8% disputes in 7 days → THROTTLE
- High Early Fraud Warning volume spike → CAUTION/THROTTLE

Acceptance Criteria

- State changes are inserted to platform_risk_state with metrics_snapshot and reason_codes.
- All payments immediately use the new limits from getEffectiveLimits().
- FREEZE blocks new creator activation until admin unfreezes.

12. Cron Jobs + SQL (Exact Queries)

12.1 Daily dispute rate (platform, 30d rolling)

```
WITH tx AS (
  SELECT COUNT(*) AS total_tx
  FROM payments
  WHERE status IN ('succeeded','review_hold','refunded','disputed')
    AND created_at >= now() - interval '30 days'
),
dp AS (
  SELECT COUNT(*) AS total_disputes
  FROM disputes
  WHERE created_at >= now() - interval '30 days'
)
SELECT (dp.total_disputes::decimal / NULLIF(tx.total_tx,0)) * 100 AS dispute_rate_pct
FROM tx, dp;
```

If dispute_rate_pct >= 0.7 then insert platform_risk_state(state='FREEZE', set_by='system', reason_codes=['PLATFORM_DISPUTE_FREEZE']).

12.2 Hourly daily-GMV spike (24h vs avg daily 7d)

```
WITH last_24h AS (
  SELECT COALESCE(SUM(amount),0) AS gmv_24h
  FROM payments
  WHERE status='succeeded'
    AND created_at >= now() - interval '24 hours'
),
avg_7d AS (
  SELECT COALESCE(SUM(amount),0)/7 AS avg_daily_7d
  FROM payments
  WHERE status='succeeded'
    AND created_at >= now() - interval '7 days'
)
SELECT last_24h.gmv_24h, avg_7d.avg_daily_7d,
       CASE WHEN avg_7d.avg_daily_7d=0 THEN 0
            ELSE last_24h.gmv_24h::decimal/avg_7d.avg_daily_7d END AS ratio
FROM last_24h, avg_7d;
```

If ratio >= 1.5 then set CAUTION (or THROTTLE if ratio >= 2.0).

12.3 Weekly GMV spike (last 7d vs previous 7d)

```
WITH this_week AS (
  SELECT COALESCE(SUM(amount),0) AS gmv
  FROM payments
  WHERE status='succeeded'
    AND created_at >= now() - interval '7 days'
),
prev_week AS (
  SELECT COALESCE(SUM(amount),0) AS gmv
  FROM payments
  WHERE status='succeeded'
    AND created_at >= now() - interval '14 days'
    AND created_at <  now() - interval '7 days'
)
SELECT this_week.gmv AS gmv_this_week, prev_week.gmv AS gmv_prev_week,
       CASE WHEN prev_week.gmv=0 THEN 0 ELSE this_week.gmv::decimal/prev_week.gmv END AS ratio
FROM this_week, prev_week;
```

If ratio >= 1.3 then set THROTTLE.

12.4 Creators above 0.8% disputes (30d) + count >= 5

```
WITH creator_tx AS (
  SELECT creator_id, COUNT(*) AS tx
  FROM payments
  WHERE created_at >= now() - interval '30 days'
    AND status IN ('succeeded','review_hold','refunded','disputed')
  GROUP BY creator_id
```

```
),
creator_dp AS (
  SELECT creator_id, COUNT(*) AS disputes
  FROM disputes
  WHERE created_at >= now() - interval '30 days'
  GROUP BY creator_id
),
rates AS (
  SELECT t.creator_id,
         (COALESCE(d.disputes,0)::decimal / NULLIF(t.tx,0)) * 100 AS dispute_rate_pct
  FROM creator_tx t
  LEFT JOIN creator_dp d ON d.creator_id = t.creator_id
)
SELECT COUNT(*) AS creators_over_08
FROM rates
WHERE dispute_rate_pct >= 0.8;
```

If creators_over_08 >= 5 then set THROTTLE and raise reserve floors.

13. Stripe Webhooks (Handlers + What to Do)

Must handle these events

- payment_intent.succeeded → mark payment succeeded; update rollups
- charge.refunded (or refund events) → mark refund; update metrics
- charge.dispute.created → create dispute record; trigger reserve update; generate evidence bundle task
- charge.dispute.closed → update dispute status; re-calc creator metrics
- early_fraud_warning.created → create EFW record; notify creator (refund recommended); add to morning queue

Webhook signature verification (Node.js example)

```
const endpointSecret = process.env.STRIPE_WEBHOOK_SECRET;

app.post('/webhooks/stripe', express.raw({type: 'application/json'}), (req, res) => {
  const sig = req.headers['stripe-signature'];
  let event;
  try {
    event = stripe.webhooks.constructEvent(req.body, sig, endpointSecret);
  } catch (err) {
    return res.status(400).send(`Webhook Error: ${err.message}`);
  }

  // idempotency: check event.id processed
  // handle event types...
  res.json({received: true});
});
```

Acceptance Criteria

- Handlers are idempotent (event.id stored; duplicates ignored safely).
- Each event updates internal ledger + rollups + creator_metrics.
- Dispute events create immediate dashboard items for founder.

14. Pop-up / Banner Copy Library (Supporter + Creator)

Supporter popups (simple English)

- **STEP_UP_REQUIRED** — Title: Confirm Your Payment • Body: For your security, please confirm this payment.
- **COOLDOWN_ACTIVE** — Title: Please Wait • Body: You are paying too fast. Please wait 15 minutes and try again.
- **NEW_CREATOR_LIMIT** — Title: Limit Reached • Body: You can support 1 new creator per day for safety. You can still support creators you paid before.
- **NEW_CREATOR_RESTRICTED** — Title: Safety Limit • Body: New creator payments are temporarily limited. You can still pay creators you already supported.
- **REVIEW_HOLD** — Title: Payment Processing • Body: Your payment is confirmed. It may take a little longer to process for safety.
- **THROTTLE_LIMITS_ACTIVE** — Title: Safety Limits Active • Body: We are temporarily limiting payments to protect supporters and creators.

Creator banners

- **RESERVE_APPLIED** — A temporary reserve has been applied due to increased disputes. This protects payouts long■term.
- **PAYOUT_DELAYED** — Your payout is delayed for a short safety review.
- **ONBOARDING_PAUSED** — New creator onboarding is temporarily paused for payment safety.
- **REFUND_RECOMMENDED** — This payment may be disputed. Consider refunding to avoid a chargeback.

Acceptance Criteria

- Every risk decision returns a message_key and message copy for UI.
- Every popup shown is logged in audit_logs with message_key and context.

15. Friday Payout Engine (Full Requirements)

Objective

- Automate weekly payouts with safety buffer: exclude review_hold, apply reserves, subtract refunds/disputes.
- Avoid platform cash loss from post-payout disputes.

Payout calculation (per creator)

```
net_payout =
  sum(succeeded payments)
- sum(refunds)
- sum(disputes)
- reserve_hold (reserve_percent * eligible_amount)
- review_hold (excluded entirely)
+/- previous_negative_balance (carried)

If net_payout < 0:
  set creator negative balance = abs(net_payout)
  payout = 0
```

Build requirements

- Create payout preview endpoint and execute endpoint.
- Persist payout_runs with totals (by creator + platform total).
- Log every payout action in audit_logs (preview + execute).
- If platform state is THROTTLE/FREEZE: allow payout delays per creator (payout_delay_days).

Testing requirements

- Test preview matches execute (same totals) using fixed dataset.
- Test review_hold payments are excluded.
- Test reserves are deducted correctly at 5/10/15%.
- Test negative balances carry forward.

Acceptance Criteria

- No review_hold payments are paid out.
- Reserve percent is applied exactly as stored in creator_metrics.
- Payout run is fully auditable from logs.

16. Creator Onboarding Throttle (Activation Batching)

Objective

- You plan 150 creators (March), 500 (April), then 500/month.
- Activation must be paced so spikes do not trigger Stripe monitoring.
- Onboarding must automatically slow down when platform is CAUTION/THROTTLE and stop in FREEZE.

Rules

- NORMAL: max 25 activations/day
- CAUTION: max 10 activations/day
- THROTTLE: max 5 activations/day
- FREEZE: 0 activations/day (blocked)

Build requirements

- Add creator_activation_state: PENDING / ACTIVE / PAUSED.
- Activation endpoint must check platform_risk_state and daily activation count.
- Daily activation counter resets at 00:00 UTC.
- Founder dashboard shows activations today + limit.

Testing requirements

- Set platform state to THROTTLE; attempt to activate 6 creators → 6th must fail.
- Set platform state to FREEZE; attempt activation → must fail always.

Acceptance Criteria

- Activation caps enforced correctly per state.
- All activation rejections return clear error message to admin.
- Audit log exists for each activation + rejection.

17. Top-Buyer Concentration Guardrail (Creator Risk)

Objective

- Stripe dislikes concentration (one buyer = huge % of a creator's GMV).
- We must detect and automatically apply safety actions.

Rule

- Compute top_buyer_percent per creator for last 30 days.
- If top_buyer_percent >= 40% and creator GMV >= threshold (e.g., £5,000/mo) → apply creator-level safety: step-up required for that buyer, consider review_hold for high values, increase reserve percent floor.

Build requirements

- Compute top buyer per creator: group by creator_id, risk_identity_id sum(amount).
- Store top_buyer_percent in creator_metrics daily.
- If rule triggers, log action and show creator banner: 'Growth pacing active'.

Testing requirements

- Create staging dataset where one identity contributes 50% GMV to a creator.
- Verify creator_metrics.top_buyer_percent updates.
- Verify safety actions apply.

Acceptance Criteria

- Top buyer concentration is visible and used by risk engine decisions.
- Actions are logged and explainable.

18. Founder Morning Dashboard (Sleep-Easy Mode)

Must display (top summary)

- Platform state (NORMAL/CAUTION/THROTTLE/FREEZE) + reason
- GMV 24h / 7d / 30d
- Platform dispute rate 30d
- Creators >= 0.5% disputes count
- Creators >= 0.8% disputes count
- Identities in cooldown count
- Payments in review_hold count + total amount
- Reserve total held estimate
- Exposure estimate (see next section)

Action Queue (things founder must review)

- New platform state changes (auto or admin)
- Early fraud warnings (EFW) in last 24h
- New disputes created in last 24h
- Creators whose reserve_percent changed
- Creators flagged for concentration risk
- Largest single-identity spenders (24h)

Acceptance Criteria

- Dashboard refreshes at least every 5 minutes (or real-time).
- Every listed item links to the underlying payment/creator/identity record.
- Founder can export action queue to CSV.

19. Exposure Estimate (Simple but Required)

Objective

- Founder must see: ‘If disputes hit this week, what is our worst-case exposure?’
- This is essential once you scale to 1,500 creators.

Definition

- Exposure estimate = (payments paid out in last 14 days) * expected dispute rate - reserves held - refunds processed
- Also show: review_hold total (reduces exposure).

Build requirements

- Compute weekly exposure numbers daily.
- Show in dashboard with trend arrow (up/down).
- If exposure exceeds threshold (configurable), system recommends THROTTLE.

Acceptance Criteria

- Exposure number is computed and visible.
- Inputs are explainable (click to see components).

20. One-Click Stripe Audit Export (Must-have at scale)

Objective

- If Stripe asks questions, we respond quickly with facts and evidence.
- Export must be generated in minutes, not days.

Endpoint

- GET /admin/export/audit-pack?range=30d (and 90d)
- Return a ZIP containing CSV + PDF summary (PDF can be simple; CSV must be accurate).

Must include

- Platform dispute metrics (30d, 90d)
- Creator dispute table (tx, disputes, dispute_rate_30d, reserve_percent)
- Refund ratio
- Top 50 identities by spend (30d)
- Confirmation logs sample (linked to payments)
- 3DS outcomes (if available from PI/charge)
- Webhook processing logs (event.id list)
- Current platform_risk_state + historical changes

Acceptance Criteria

- Export completes under 60 minutes for large datasets.
- All files are timestamped and checksummed.
- Audit log of export creation exists.

21. Stress Testing (How to Test Before Launch)

Objective

- Stress test means: simulate risky behaviour and confirm system responds automatically.
- Do this in staging with Stripe test mode.

Build: Simulation harness

- Create a script/tool: simulate_payments.js (or Python) that creates identities, creators, and runs payment patterns.
- Patterns: normal user, whale, hopper, regret/dispute injection, GMV spike, EFW spike.
- Script must output: expected decisions vs actual decisions (pass/fail).

Minimum scenarios

- S1 Acceleration: 3 payments in 10 minutes → STEP_UP
- S2 Repeat acceleration → COOLDOWN
- S3 New creator hopping → BLOCK
- S4 Whale cross-creator spend >£5k/48h → restrict new creators
- S5 GMV spike 150% daily → CAUTION
- S6 Platform dispute 0.7% → FREEZE onboarding
- S7 Payout run with review_hold + reserves + negative balances
- S8 Early fraud warning webhook → creator notified + queue item

Acceptance Criteria

- All scenarios produce correct platform state and user outcomes.
- No PaymentIntents created when rules say BLOCK/COOLDOWN.
- Payout engine excludes review_hold and applies reserves correctly.

22. Go-Live Checklist (Must be TRUE before onboarding wave)

Must be true

- Risk Engine blocks PaymentIntent creation reliably.
- Webhook handlers are verified + idempotent.
- Step-up + cooldown + review_hold are working and logged.
- Platform throttle states change automatically based on cron jobs.
- Onboarding activation batching is enforced by platform state.
- Friday payout preview + execute are tested twice with the same results.
- Audit export endpoint produces correct metrics.
- Founder dashboard shows action queue and exposure estimate.

Final Acceptance

- If any item above fails → do not onboard new creators.
- Fix, re-test, then proceed.